

CO3-LAB EXPERIMENTS

1. Prolog Program to Implement Towers of Hanoi

Program:

```
% Towers of Hanoi
```

```
hanoi(1, Source, Destination, _) :-
```

```
    write('Move disk from '),
```

```
    write(Source),
```

```
    write(' to '),
```

```
    write(Destination),
```

```
    nl.
```

```
hanoi(N, Source, Destination, Auxiliary) :-
```

```
    N > 1,
```

```
    N1 is N - 1,
```

```
    hanoi(N1, Source, Auxiliary, Destination),
```

```
    hanoi(1, Source, Destination, Auxiliary),
```

```
    hanoi(N1, Auxiliary, Destination, Source).
```

Output:

```
[1] 33 ?- % c:/users/malaravan/downloads/ai/sum compiled 0.00 sec, -8 clauses
[1] 33 ?- hanoi(3, a, c, b).
Move disk from a to c
Move disk from a to b
Move disk from c to b
Move disk from a to c
Move disk from b to a
Move disk from b to c
Move disk from a to c
true ▲
```

2. Prolog Program to Check Whether a Bird Can Fly

Program:

```
% Bird Database
```

```
bird(parrot).
```

```
bird(eagle).
```

```
bird(pigeon).
```

```
bird(hen).
```

```
bird(penguin).
```

```
can_fly(parrot).
```

```
can_fly(eagle).
```

```
can_fly(pigeon).
```

```
cannot_fly(hen).
```

```
cannot_fly(penguin).
```

```
% Rule
```

```
flies(Bird) :-
```

```
    can_fly(Bird).
```

```
does_not_fly(Bird) :-
```

```
    cannot_fly(Bird).
```

Output:

```
[1] 34 ?- flies(parrot).  
true.  
  
[1] 35 ?- flies(penguin).  
false.  
  
[1] 36 ?- flies(Bird).  
Bird = parrot ;  
Bird = eagle ;  
Bird = pigeon.
```

3. Prolog Program to Implement Family Tree

Program:

```
% Family Tree
```

```
parent(john, mary).
```

```
parent(john, david).
```

```
parent(susan, mary).
```

```
parent(susan, david).
```

```
parent(david, peter).
```

```
parent(david, lisa).
```

```
male(john).
```

```
male(david).
```

```
male(peter).
```

```
female(susan).
```

```
female(mary).
```

```
female(lisa).
```

```
% Rules
```

```
father(X, Y) :-
```

```
    male(X),
```

```
    parent(X, Y).
```

```
mother(X, Y) :-
```

```
    female(X),
```

```
    parent(X, Y).
```

grandparent(X, Y) :-

parent(X, Z),

parent(Z, Y).

sibling(X, Y) :-

parent(P, X),

parent(P, Y),

X \= Y.

Output:

```
[1] 39 ?- parent(john, Child).  
Child = mary ;  
Child = david.  
  
[1] 40 ?- father(Father, mary).  
Father = john ;  
false.  
  
[1] 41 ?- grandparent(Grandparent, peter).  
Grandparent = john ;  
Grandparent = susan ;  
false.  
  
[1] 42 ?- sibling(mary, Sibling).  
Sibling = david ;  
Sibling = david.
```

4. Prolog Program for Diet Suggestion Based on Disease

Program:

% Diet Recommendation System

diet(diabetes, 'Low sugar diet').

diet(hypertension, 'Low salt diet').

diet(obesity, 'Low calorie diet').

diet(anemia, 'Iron rich diet').

diet(fever, 'Light and nutritious food').

% Rule

suggest_diet(Disease, Diet) :-

diet(Disease, Diet).

Output:

```
[1] 43 ?- % c:/users/malaravan/downloads/ai/sum compiled 0.00 sec, -10 clause
[1] 43 ?- suggest_diet(diabetes, Diet).
Diet = 'Low sugar diet'.

[1] 44 ?- suggest_diet(hypertension, Diet).
Diet = 'Low salt diet'.
```

5. Prolog Program to Implement Monkey Banana Problem

Program:

% Monkey Banana Problem

% State:

% state(MonkeyPosition, BoxPosition, MonkeyHeight, BananaPosition)

can_get_banana(

state(Monkey, Box, floor, Banana),

state(Monkey, Box, standing, Banana)

) :-

Monkey = Box.

can_get_banana(

state(Monkey, Box, floor, Banana),

state(Box, Box, floor, Banana)

) :-

Monkey \= Box.

```
can_get_banana(  
    state(Box, Box, floor, Banana),  
    state(Box, Box, standing, Banana)  
).
```

```
can_get_banana(  
    state(Box, Box, standing, Banana),  
    state(Box, Box, standing, Banana)  
) :-  
    Box = Banana.
```

Output:

```
[1] 54 ?- solve.  
Monkey moves from door to window.  
Monkey pushes the box to the center.  
Monkey climbs the box.  
Monkey gets the banana.  
true.
```

6. Prolog Program for Fruit and Its Color Using Backtracking

Program:

% Fruit and Color Database

```
fruit_color(apple, red).  
fruit_color(banana, yellow).  
fruit_color(orange, orange).  
fruit_color(grapes, green).  
fruit_color(mango, yellow).  
fruit_color(guava, green).
```

Output:

```
[1] 55 ?- % c:/users/malaravan/downloads/ai/sum compiled 0.00 sec, -2 clauses
[1] 55 ?- fruit_color(Fruit, Color).
Fruit = apple,
Color = red ;
Fruit = banana,
Color = yellow ;
Fruit = Color, Color = orange ;
Fruit = grapes,
Color = green ;
Fruit = mango,
Color = yellow ;
Fruit = guava,
Color = green.

[1] 56 ?- fruit_color(Fruit, yellow).
Fruit = banana .
```

7. Prolog Program to Implement Best First Search

Program:

% Best First Search

edge(a, b).

edge(a, c).

edge(b, d).

edge(b, e).

edge(c, f).

edge(c, g).

% Heuristic values

h(a, 5).

h(b, 4).

h(c, 3).

h(d, 1).

h(e, 2).

h(f, 1).

h(g, 0).

best_first(Start, Goal, Path) :-

search([[Start]], Goal, Path).

search([[Goal|Path]|_], Goal, Result) :-

reverse([Goal|Path], Result).

search([[Current|Path]|Rest], Goal, Result) :-

findall(

[Next,Current|Path],

(

edge(Current, Next),

\+ member(Next, [Current|Path])

),

Children

),

add_to_queue(Children, Rest, NewQueue),

sort_queue(NewQueue, SortedQueue),

search(SortedQueue, Goal, Result).

add_to_queue([], Queue, Queue).

add_to_queue([H|T], Queue, NewQueue) :-

append(Queue, [H], Temp),

add_to_queue(T, Temp, NewQueue).

sort_queue(Queue, Sorted) :-

predsort(compare_paths, Queue, Sorted).

compare_paths(Order, [A|_], [B|_]) :-

h(A, HA),

h(B, HB),

compare(Order, HA, HB).

Output:

```
[1] 66 ?- best_first(a, f, Path), write(Path), nl.  
[a,c,f]  
Path = [...|...] .  
  
[1] 67 ?- best_first(a, g, Path), write(Path), nl.  
[a,c,g]  
Path = [...|...] .  
  
[1] 68 ?- best_first(a, d, Path), write(Path), nl.  
[a,b,d]  
Path = [...|...] .
```

8. Prolog Program for Medical Diagnosis

Program:

% Medical Diagnosis System

symptom(ravi, fever).

symptom(ravi, cough).

symptom(priya, fever).

symptom(priya, rash).

symptom(arun, cough).

symptom(arun, breathlessness).

% Diagnosis rules

diagnosis(Patient, flu) :-

```
symptom(Patient, fever),  
symptom(Patient, cough).
```

diagnosis(Patient, measles) :-

```
symptom(Patient, fever),  
symptom(Patient, rash).
```

diagnosis(Patient, pneumonia) :-

```
symptom(Patient, cough),  
symptom(Patient, breathlessness).
```

Output:

```
[1] 58 ?- % c:/users/malaravan/downloads/ai/sum compiled 0.00 sec, -11 clauses  
[1] 58 ?- diagnosis(ravi, Disease).  
Disease = flu .  
  
[1] 59 ?- diagnosis(arun, Disease).  
Disease = pneumonia.  
  
[1] 60 ?- ▲
```

9. Prolog Program for Forward Chaining

Program:

```
% Forward Chaining
```

```
fact(fever).
```

```
fact(cough).
```

```
rule(fever, flu).
```

```
rule(cough, cold).
```

forward_chaining :-

```
fact(Fact),
```

```
rule(Fact, Conclusion),  
write(Fact),  
write(' -> '),  
write(Conclusion),  
nl,  
fail.
```

forward_chaining.

Output:

```
[1] 60 ?- % c:/users/malaravan/downloads/ai/sum compiled 0.00 sec, -3 clauses  
[1] 60 ?- forward_chaining.  
fever -> flu  
cough -> cold  
true.  
  
[1] 61 ?- rule(fever, Disease).  
Disease = flu.
```

10. Prolog Program for Backward Chaining

Program:

% Backward Chaining

fact(fever).

fact(cough).

rule(flu, fever).

rule(flu, cough).

% Backward chaining

prove(Goal) :-

fact(Goal).

prove(Goal) :-

rule(Goal, Condition1),

prove(Condition1),

rule(Goal, Condition2),

prove(Condition2).

Output:

```
[1] 62 ?- % c:/users/malaravan/downloads/ai/sum compiled 0.00 sec, 0 clauses
% c:/users/malaravan/downloads/ai/sum compiled 0.00 sec, -1 clauses
[1] 62 ?- flu.
true.

[1] 63 ?- fever.
true.
```